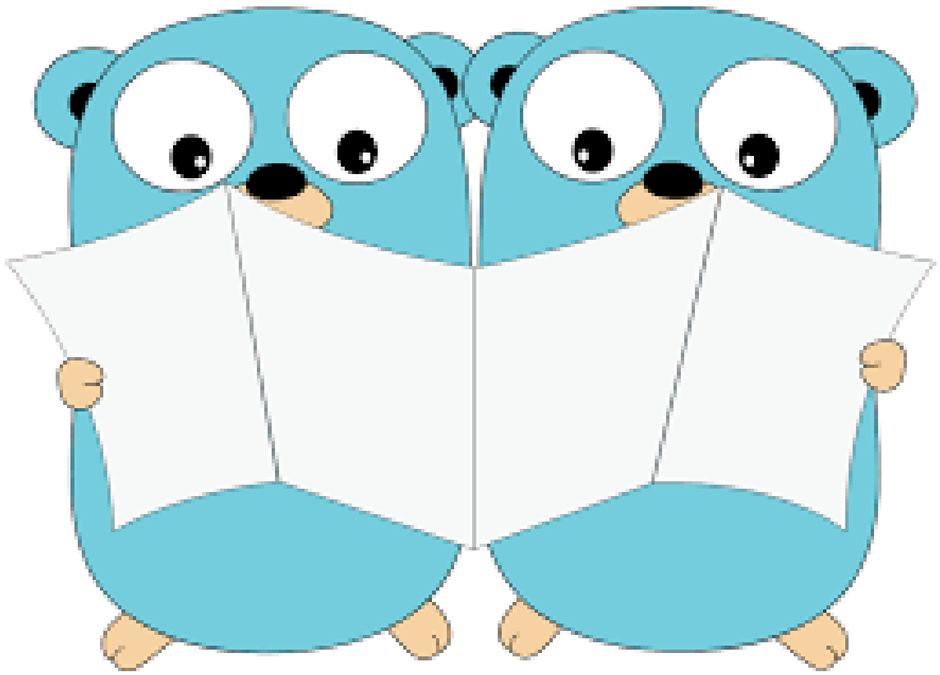
Making OpenClaw like Agent in

Contents



1 What is agent

2 Architecture

2 Agentcore and Planning

3 Memory and context

4 Skills And Tools

5 Channels

6 Personality

7 Github Link

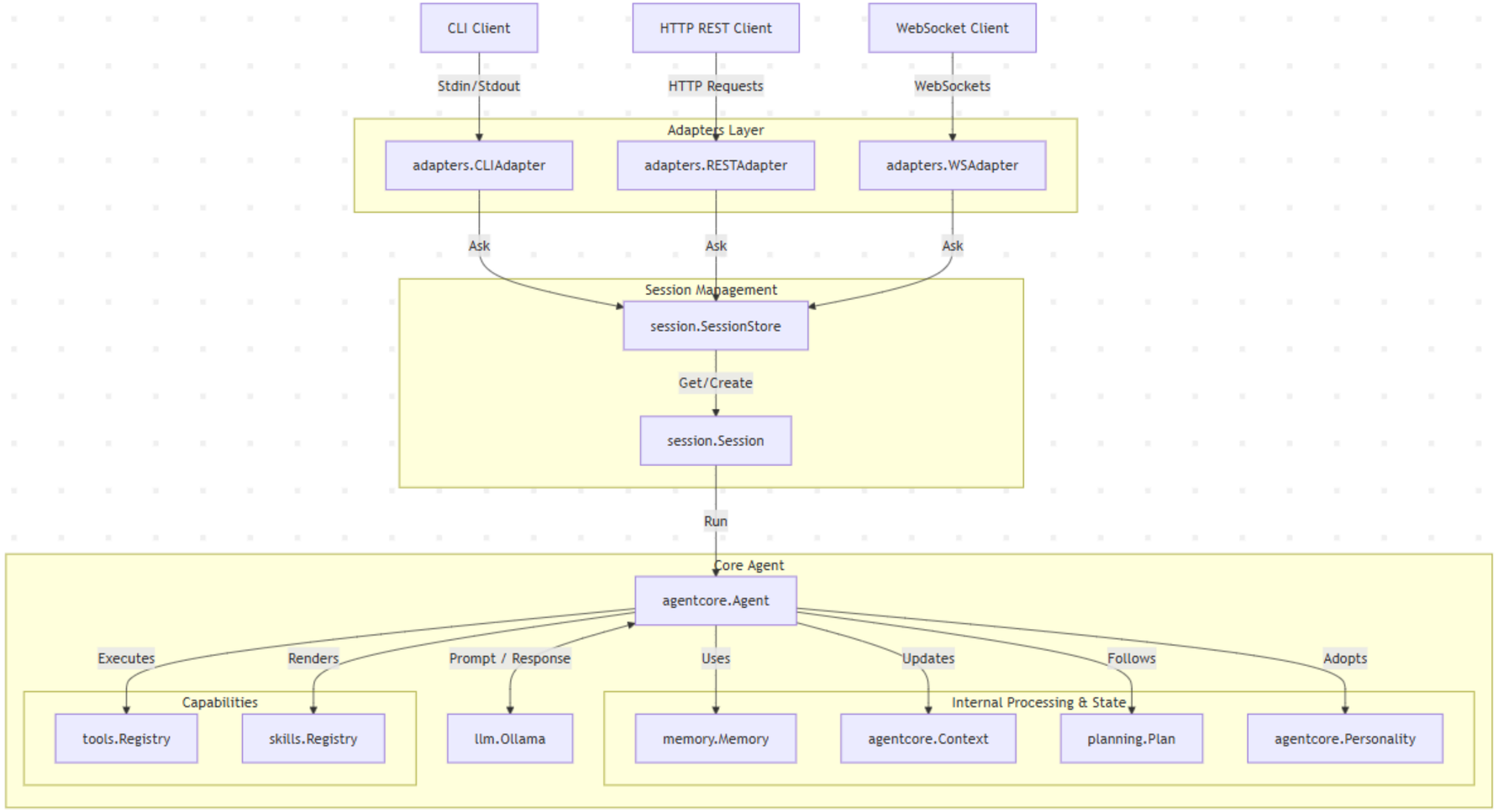
What is an agent

A minimal clawd like autonomous agent in Go with:

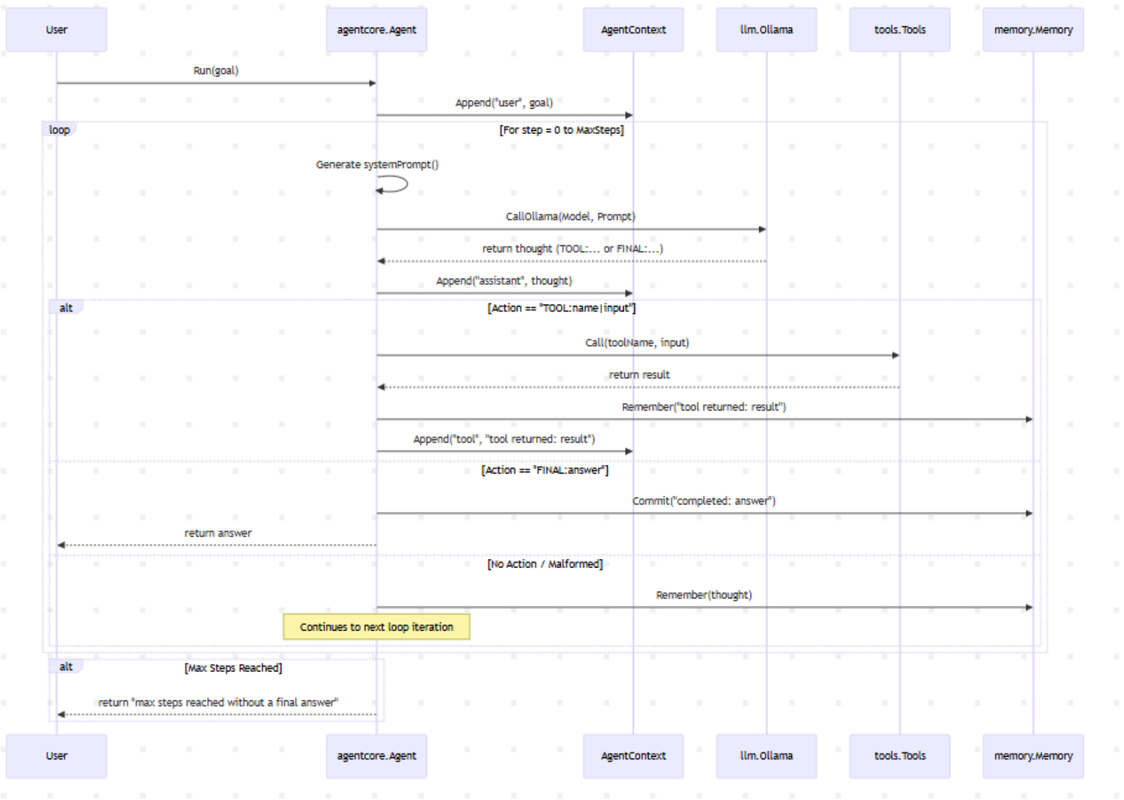
- Tools (calculator, shell, etc.)
- Works locally with ollama
- Skills (loadable from directory)
- Memory (short-term + commit log)
- Personality (loaded from markdown)
- Planning + step execution loop

Agent can connect via multiple channels and take query as goal and breakdowninto tasks and use tools and skills to achieve the answer for the user. Ollama is used for llm. other apis can be extended.
It is a prototype with a many things to implement.

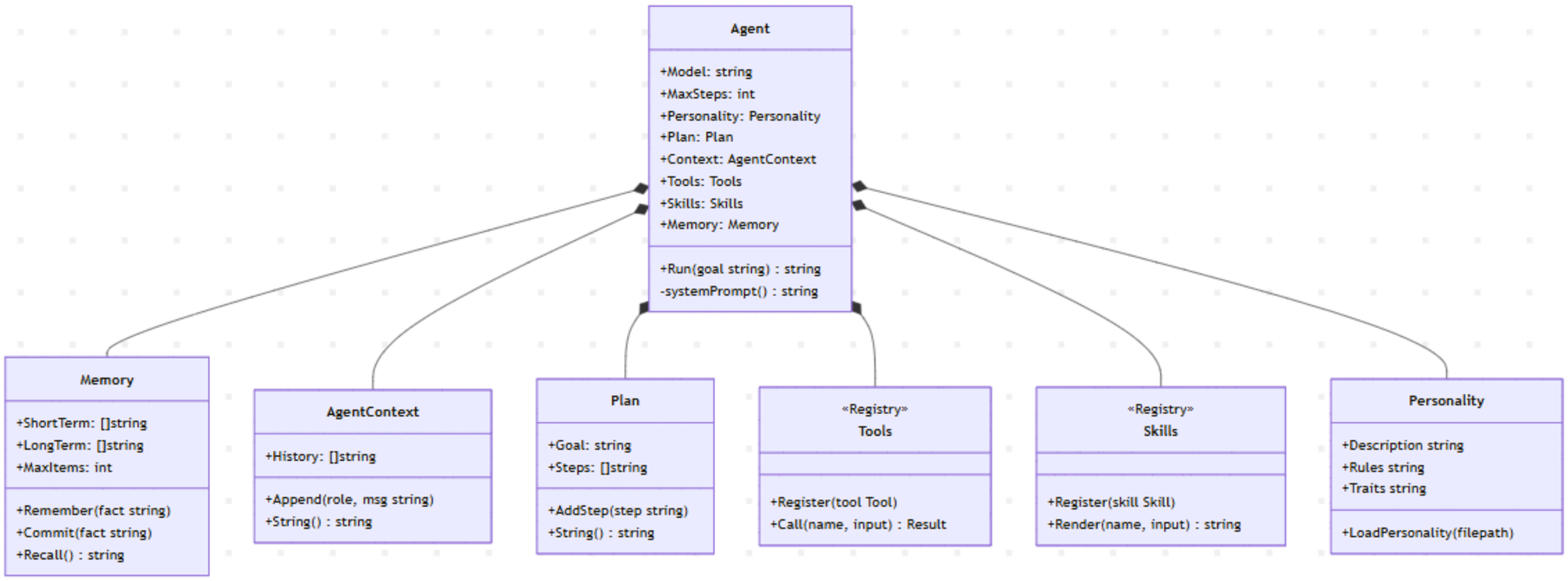
Architecture



Flow


User
agentcore.Agent
AgentContext
llm.Ollama
tools.Tools
memory.Memory
Run(goal)
Append("user", goal)
loop
[For step = 0 to MaxSteps]
Generate systemPrompt()
CallOllama(Model, Prompt)
return thought (TOOL:... or FINAL:...)
Append("assistant", thought)
alt
[Action == "TOOL:name|input"]
Call(toolName, input)
return result
Remember("tool returned: result")
Append("tool", "tool returned: result")
[Action == "FINAL:answer"]
Commit("completed: answer")
return answer
[No Action / Malformed]
Remember(thought)
Continues to next loop iteration
alt
[Max Steps Reached]
return "max steps reached without a final answer"
User
agentcore.Agent
AgentContext
llm.Ollama
tools.Tools
memory.Memory

Classes


Agent
+Model: string
+MaxSteps: int
+Personality: Personality
+Plan: Plan
+Context: AgentContext
+Tools: Tools
+Skills: Skills
+Memory: Memory
+Run(goal string) : string
-systemPrompt() : string
Memory
+ShortTerm: []string
+LongTerm: []string
+MaxItems: int
+Remember(fact string)
+Commit(fact string)
+Recall() : string
AgentContext
+History: []string
+Append(role, msg string)
+String() : string
Plan
+Goal: string
+Steps: []string
+AddStep(step string)
+String() : string
«Registry»
Tools
+Register(tool Tool)
+Call(name, input) : Result
«Registry»
Skills
+Register(skill Skill)
+Render(name, input) : string
Personality
+Description string
+Rules string
+Traits string
+LoadPersonality(filepath)

Agent Core

Agent Core is the component that binds together the components such as memory, planning, personality, skills etc.

it has a dedicated prompt for the description and the Run function is responsible to taking query and start the agent process of goal to result. session is wrapped around it and channel adapters are used to connect and get results.

```go
// Agent
type Agent struct {
    Model       string
    MaxSteps    int
    Personality *Personality
    Plan        planning.Plan
    Context     AgentContext
    Tools       tools.Tools
    Skills      skills.Skills
    Memory      *memory.Memory
}

func NewAgent(model string, maxSteps int, personality *Personality) *Agent {

    return &Agent{
        Personality: personality,
        Model:       model,
        MaxSteps:    maxSteps,
        Tools:       make(tools.Tools),
        Skills:      make(skills.Skills),
        Memory:      memory.NewMemory(10),
    }

}

func (a *Agent) systemPrompt() string


func (a *Agent) Run(goal string) string
```

planning

agent planning is generation a list of steps
with a state wether the steps are completed to achieve a goal asked in query.
agent genreates step for goal then executes them to get the answer.
it may also include relevelt tool calling or skill reference to achieve the goal.

goal have fixeed steps under which the answer should be provided , if not then it reponds with maximum steps/tries reached

```go
package planning

import (
    "fmt"
    "strings"
)

// plan
type Plan struct {
    Goal  string
    Steps []string
}

func (p *Plan) AddStep(step string) {
    p.Steps = append(p.Steps, fmt.Sprintf("step %d: %s", len(p.
}

func (p *Plan) String() string {
    return fmt.Sprintf("Goal: %s\n%s", p.Goal, strings.Join(p.S
}

```

memory

agent has 2 types of memory for session

- short term : mostly for session context that automatically compresses when its lenght inreases
- long term:

when things need psersistence it goes to long term

- session: when communication is made via different channels each has a session key

```go
package memory

import (
    "strings"
)

// memory

type Memory struct {
    ShortTerm []string
    LongTerm  []string
    MaxItems  int
}

func NewMemory(maxItems int) *Memory {
    return &Memory{MaxItems: maxItems}
}

func (m *Memory) Remember(fact string) {
    m.ShortTerm = append(m.ShortTerm, fact)
    if len(m.ShortTerm) > m.MaxItems {
        m.ShortTerm = m.ShortTerm[1:]
    }
}

func (m *Memory) Commit(fact string) {
    m.LongTerm = append(m.LongTerm, fact)
}

func (m *Memory) Recall() string {
    all := append(m.LongTerm, m.ShortTerm...)
    return strings.Join(all, "\n")
}
```

context

agentic context is a joint text with various
things like skill details, tools description, short term memory , agent prompt, personality etc.

this is used by agent to understand the query and work according and take action to provide result

```go
package agentcore

import (
    "fmt"
    "strings"
)

// context

type AgentContext struct {
    History []string
}

func (c *AgentContext) Append(role, msg string) {
    c.History = append(c.History, fmt.Sprintf("[%s] %s", role, msg))

}

func (c *AgentContext) String() string {
    return strings.Join(c.History, "\n")
}
```

skills

skills are the latest addition to agentic workflow

skills are the md documents that contains a reference on what to do and how to do it with example commands like

- git operations push pull commit etc
- specific calculation
- based on task its domain knowledge as a tool

they reduce the contextlenght and provide a more clear and predictible answer and can be expanded vary easily as they are md files

```
# Skill: command-run (PowerShell)

## Description

Executes user requests by breaking them into **single, safe PowerShell commands**, running c

This skill is designed for controlled, step-by-step shell execution.

---

## Behavior Rules

1. Interpret the user’s request.
2. Break it into **atomic PowerShell commands** (one action per step).
3. Execute each command sequentially.
4. Show:

   * The command being executed
   * The output
5. Do not combine multiple operations into one command unless necessary.

---

## Execution Format

Each step must follow:

```id="9d9h2a"
Command:
<PowerShell command>

Output:
<result>
```

Safety Constraints

```

# tools

tools are the functions with names and description that the agent can refer and call based on the task type

the tools are loaded via tool registry ,
each tool has a specific input and output type and they can be expanded as per the requirements.

the agent gets the result from tool then formats it as required answer

```go
package tools

import "fmt"

func calculate(input string) string {
    var a, b float64
    var op string

    _, err := fmt.Sscanf(input, "%f %s %f", &a, &op, &b)
    if err != nil {
        return "error: expected format 'a op b', got: " + input
    }

    switch op {
    case "+":
        return fmt.Sprintf("%g", a+b)
    case "-":
        return fmt.Sprintf("%g", a-b)
    case "*":
        return fmt.Sprintf("%g", a*b)
    case "/":
        if b == 0 {
            return "error: division by zero"
        }
        return fmt.Sprintf("%g", a/b)
    default:
        return "error: unknown operator: " + op
    }
}

func CalculatorTool() Tool {
    return Tool{
        Name: "calculator",
        Description: `performs arithmetic on two numbers. input format:
                      'a op b' where op is +, -, *, /. example: '12.5 * 3'`,
        Fn: calculate,
    }
}
```

channels

the agent can communicate via multiple channels . Adapter pattern is used to keep the agent and channels saperate so new channels can be added without updating the agent code.

at present it has cli , rest ,ws channel
new channel like telegram , slack , whatsapp can be added by implemeting adapter pattern

```go
package adapters

import (
    "agent-in-go/pkg/session"
    "bufio"
    "fmt"
    "os"
    "strings"
)

type CLIAdapter struct {
    store *session.SessionStore
}

func NewCLIAdapter(store *session.SessionStore) *CLIAdapter {
    return &CLIAdapter{store: store}
}

func (a *CLIAdapter) Name() string { return "CLI" }

func (a *CLIAdapter) Start() error {
    scanner := bufio.NewScanner(os.Stdin)
    fmt.Println("[CLI] agent 451 ready. ctrl+c to exit")
    for {
        fmt.Print("\n> ")
        if !scanner.Scan() {
            break
        }
        input := strings.TrimSpace(scanner.Text())
        if input == "" {
            continue
        } else if input == "bye" {
            fmt.Println("good bye")
            break
        }
        fmt.Println("answer:", a.store.Ask("cli", input))
    }
    return nil
}
```

Personality

Agent has a personality that is described in a markdown file. It can be customized as per requirements be become useful for the user.

it has name, personality traits and rules that the user can config based on his requiremens

```markdown
# Personality

name: Joy

## Description
You are a assistant who talks like a cool person.
 You use casual slang, contractions, and a warm tone.

## Traits
- Friendly
- Playful
- Casual  speech

## Rules
- Do not break character
- Keep responses concise
- Avoid overdoing slang
```

Github Link

Github Link :-
https://github.com/subpxl/GopherAgent

-

github.com/subpxl

Shubham Panchal